Relatório do Projeto AED

Tiago Pedrosa : 116113
Gustavo Rodrigues : 126033

Algoritmos e Estruturas de Dados
Universidade de Aveiro
Engenharia de Computadores e Informática

Conteúdo

1 Introdução **3**
1.1 Objetivos 3

2 Estruturas de Dados e Funções Básicas **3**
2.1 Estrutura da Imagem 3
2.2 Funções Implementadas 3
2.2.1 ImageCopy 3
2.2.2 ImageRotate90 e ImageRotate180 3

3 Análise Detalhada da Função ImageIsEqual **4**
3.1 Análise Formal da Complexidade 4
3.2 Análise Experimental 4
3.3 Comparação: Análise Experimental vs Formal 4

4 Algoritmos de Preenchimento de Regiões **5**
4.1 Descrição dos Algoritmos 5
4.1.1 Versão Recursiva (ImageRegionFillingRecursive) 5
4.1.2 Versão com Stack Explícito (ImageRegionFillingWithSTACK) . . . 5
4.1.3 Versão com Fila (ImageRegionFillingWithQUEUE) 5
4.2 Análise de Complexidade 5
4.3 Comparação das Estratégias 5
4.3.1 Versão Recursiva 5
4.3.2 Versão com Stack 6
4.3.3 Versão com Fila 6
4.4 Resultados Experimentais 6

5 Segmentação de Imagens **6**

6 Conclusões **7**

1 Introdução

Este relatório apresenta o desenvolvimento do trabalho prático de Algoritmos e Estruturas de Dados, que consiste na implementação de um módulo para processamento de imagens RGB utilizando uma tabela de cores (LUT).

O projeto utiliza uma representação eficiente de imagens RGB onde cada pixel armazena um índice para uma tabela de cores. Esta abordagem é vantajosa quando o número de cores distintas na imagem é significativamente menor que o número total de pixels.

1.1 Objetivos

Este trabalho tem como objetivos implementar funções de manipulação de imagens e desenvolver algoritmos de preenchimento de regiões utilizando abordagens diferentes. Inclui também a implementação de segmentação de imagens baseada em regiões conectadas e análise da complexidade computacional das funções desenvolvidas.

2 Estruturas de Dados e Funções Básicas

2.1 Estrutura da Imagem

A estrutura da imagem RGB com LUT é definida da seguinte forma:

```
struct image {
    uint32 width;         // Largura da imagem
    uint32 height;        // Altura da imagem
    uint16** image;       // Matriz de labels (indices para LUT)
    uint16 num_colors;    // Numero de cores utilizadas
    rgb_t* LUT;           // Look-Up Table (tabela de cores)
};
```

2.2 Funções Implementadas

2.2.1 ImageCopy

A função `ImageCopy` cria uma cópia de uma imagem. O algoritmo aloca uma nova estrutura com as mesmas dimensões, copia a LUT completa e depois copia todos os pixels linha por linha. A complexidade temporal é $O(W \times H + C)$, onde W é a largura, H é a altura e C é o número de cores.

2.2.2 ImageRotate90 e ImageRotate180

A rotação de 90° transforma um pixel em (u, v) para $(H - 1 - v, u)$ na nova imagem, invertendo as dimensões. A rotação de 180° transforma um pixel em (u, v) para $(W - 1 - u, H - 1 - v)$ mantendo as dimensões. Ambas apresentam complexidade $O(W \times H)$.

3 Análise Detalhada da Função ImageIsEqual

3.1 Análise Formal da Complexidade

A função `ImageIsEqual` verifica se duas imagens são iguais comparando as cores RGB de todos os pixels. O algoritmo verifica inicialmente se as dimensões são iguais; posteriormente compara pixel a pixel até encontrar uma diferença ou terminar a varredura.

Para duas imagens de dimensões $W \times H$, a análise de complexidade é:

- **Melhor caso:** $\Theta(1)$ - ocorre quando as dimensões das imagens são diferentes, terminando imediatamente sem comparação de pixels.
- **Pior caso:** $\Theta(W \times H)$ - ocorre quando as imagens são completamente iguais, sendo necessário comparar todos os $W \times H$ pixels até ao fim. Cada comparação envolve 2 acessos aos arrays de pixels e comparação das cores RGB.
- **Caso médio:** $O(W \times H)$ - assumindo imagens com dimensões iguais mas conteúdo potencialmente diferente. Se as imagens diferem, a função termina quando encontra a primeira diferença. Estatisticamente, espera-se que a diferença ocorra em média após verificar metade dos pixels, mas em notação assintótica o fator constante é irrelevante.

Em termos de acessos à memória (PIXMEM), o pior caso requer $2 \times W \times H$ acessos aos pixels (uma leitura em cada imagem por comparação). A otimização early exit permite reduzir significativamente este número quando diferenças são encontradas cedo.

3.2 Análise Experimental

Realizaram-se testes computacionais com imagens de diferentes tamanhos e conteúdos, registando o número de acessos PIXMEM:

Teste	**Dimensão**	**Conteúdo**	**PIXMEM**	**Fórmula**
1	100×100	Xadrez (iguais)	20,000	$2 \times 100 \times 100$
2	24×7	feep.pbm (iguais)	336	$2 \times 24 \times 7$
3	50×50	Branca (iguais)	5,000	$2 \times 50 \times 50$
4	40×60	Xadrez (iguais)	4,800	$2 \times 40 \times 60$

Tabela 1: Resultados experimentais da função ImageIsEqual

3.3 Comparação: Análise Experimental vs Formal

Os resultados experimentais confirmam perfeitamente a análise formal. Para imagens iguais, o número de acessos PIXMEM corresponde exatamente à fórmula $2 \times W \times H$, validando a complexidade $\Theta(W \times H)$ do pior caso.

4 Algoritmos de Preenchimento de Regiões

4.1 Descrição dos Algoritmos

Implementaram-se três versões do algoritmo flood-fill para preenchimento de regiões conexas com conectividade de 4 vizinhos:

4.1.1 Versão Recursiva (ImageRegionFillingRecursive)

O algoritmo verifica se o pixel é válido e tem a cor original. Para cada pixel válido, pinta o pixel com a nova cor e chama recursivamente a função para os 4 vizinhos (cima, baixo, esquerda e direita).

4.1.2 Versão com Stack Explícito (ImageRegionFillingWithSTACK)

O algoritmo cria um stack e empilha o pixel inicial. Enquanto o stack não estiver vazio, desempilha um pixel, verifica se é válido e tem a cor original, pinta o pixel com a nova cor e empilha os 4 vizinhos.

4.1.3 Versão com Fila (ImageRegionFillingWithQUEUE)

O algoritmo cria uma fila e enfileira o pixel inicial. Enquanto a fila não estiver vazia, retira um pixel, verifica se é válido e tem a cor original, pinta o pixel com a nova cor e enfileira os 4 vizinhos.

4.2 Análise de Complexidade

Algoritmo	**Tempo**	**Espaço**	**Ordem de Preenchimento**
Recursivo	$O(N)$	$O(N)$	Profundidade (stack do sistema)
STACK	$O(N)$	$O(N)$	Profundidade (stack explícito)
QUEUE	$O(N)$	$O(N)$	Largura (níveis sucessivos)

Tabela 2: Comparação dos algoritmos de flood-fill

Todos os algoritmos visitam exatamente N pixels (onde N é o número de pixels na região) e realizam $O(1)$ operações por pixel, resultando em $O(N)$ operações totais. A complexidade espacial é $O(N)$ em todos os casos: na versão recursiva, a profundidade máxima da pilha de recursão é $O(N)$; nas versões iterativas, a estrutura de dados (stack ou fila) armazena no máximo $O(N)$ elementos.

4.3 Comparação das Estratégias

4.3.1 Versão Recursiva

Vantagens: simplicidade e clareza. Desvantagens: risco de stack overflow para regiões grandes (a pilha de sistema tem espaço limitado) e overhead de chamadas de função recursiva.

4.3.2 Versão com Stack

Vantagens: elimina o risco de stack overflow do sistema, mesmo para regiões grandes; permite alocação dinâmica de memória. Desvantagens: mais código, overhead de gerenciamento de stack.

4.3.3 Versão com Fila

Vantagens: preenchimento em camadas sucessivas a partir do ponto inicial, útil para análise de propagação. Desvantagens: maior consumo de memória em regiões muito alongadas.

4.4 Resultados Experimentais

Teste	**Algoritmo**	**Pixels Preench.**	**PIXMEM**	**Tempo (μs)**
Imagem 50×50 branca	Recursivo	2.500	12.302	163
	STACK	2.500	12.302	206
	QUEUE	2.500	12.302	188
feep.ppm	Recursivo	118	530	9
	STACK	118	530	17
	QUEUE	118	530	17
maze.pbm	Recursivo	801	4.005	39
	STACK	801	4.005	99
	QUEUE	801	4.005	83

Tabela 3: Resultados dos testes de preenchimento de regiões

Os três algoritmos produziram resultados idênticos em termos de número de acessos à memória (PIXMEM) e pixels preenchidos, confirmando que todas as três implementações estão corretas e apresentam desempenho equivalente em termos de operações de memória.

Relativamente aos tempos de execução observados, a versão recursiva foi consistentemente a mais rápida em todas as imagens testadas. Na imagem 50×50 branca obteve 163 μs, seguida pela versão com fila (188 μs) e pela versão com stack (206 μs). Este comportamento deve-se ao overhead associado às operações de push/pop em estruturas de dados explícitas comparado com a pilha de sistema. Para a imagem feep.ppm, a versão recursiva foi significativamente mais rápida (9 μs) que as versões com estruturas explícitas (17 μs cada). O teste com maze.pbm preencheu 801 pixels do labirinto, com a versão recursiva executando em 39 μs, fila em 83 μs e stack em 99 μs, mantendo o padrão observado de que a versão recursiva é a mais eficiente em termos de tempo de execução.

5 Segmentação de Imagens

A função `ImageSegmentation` identifica e rotula todas as regiões brancas de uma imagem com cores distintas. O algoritmo percorre sequencialmente todos os pixels da imagem. Quando encontra um pixel branco ainda não identificado, gera uma nova cor, aloca-a na LUT, aplica uma função de preenchimento (passada como parâmetro) para colorir a região inteira, e incrementa o contador de regiões. A complexidade temporal é $O(W \times H)$,

Teste	Algoritmo	Regiões	PIXMEM	Tempo (μs)	Cores LUT
Xadrez 60×60	Recursivo	4	11,528	88	6
Xadrez 60×60	STACK	4	11,528	140	6
Xadrez 60×60	QUEUE	4	11,528	134	6
feep.pbm	Recursivo	2	710	10	4
feep.pbm	STACK	2	710	12	4
feep.pbm	QUEUE	2	710	13	4
maze.pbm	Recursivo	1	5,686	48	3
maze.pbm	STACK	1	5,686	75	3
maze.pbm	QUEUE	1	5,686	81	3

Tabela 4: Resultados da segmentação de imagens com diferentes estratégias

pois cada pixel é visitado exatamente uma vez no loop principal e pintado no máximo uma vez durante os preenchimentos.

Os três métodos de preenchimento produziram resultados idênticos em todas as métricas. A LUT final contém as 2 cores originais (branco e preto) mais novas cores geradas para cada região branca identificada. Os valores de PIXMEM confirmam a complexidade $O(W \times H)$ da análise teórica.

A imagem maze.pbm (41×41 pixels) representa um labirinto. A segmentação identificou 1 região branca conectada (o caminho do labirinto), com 5,686 acessos PIXMEM em todos os algoritmos, correspondendo à varredura da imagem completa e ao preenchimento da região identificada.

Analisando os tempos de execução da segmentação, observa-se que a versão recursiva foi consistentemente a mais rápida em todos os testes. No xadrez 60×60 obteve 88 μs, comparado com 140 μs (stack) e 134 μs (fila). Para a imagem feep.pbm, a versão recursiva executou em 10 μs, enquanto as versões stack e fila demoraram 12 μs e 13 μs respetivamente. No labirinto maze.pbm, a versão recursiva foi significativamente mais rápida (48 μs) que stack (75 μs) e fila (81 μs), confirmando que a versão recursiva mantém vantagem de desempenho em todas as escalas de operação.

6 Conclusões

Implementaram-se com sucesso todas as funções requisitadas para manipulação de imagens RGB com LUT. As funções de cópia, comparação e rotação apresentam complexidade ótima $O(W \times H)$, sendo impossível fazer melhor considerando que todos os pixels devem ser processados.

A análise detalhada da função `ImageIsEqual` validou experimentalmente as previsões teóricas, confirmando complexidade $\Theta(W \times H)$ no pior caso.

Os algoritmos de preenchimento de regiões (flood-fill) foram implementados com sucesso em três variantes. Apesar de apresentarem a mesma complexidade $O(N)$, distinguem-se significativamente em características práticas: a versão recursiva é mais elegante mas com risco de stack overflow; a versão com stack oferece segurança e controlo; a versão com fila oferece uma perspetiva de preenchimento em largura. A seleção do algoritmo apropriado depende do contexto e requisitos.

A função de segmentação integra estes algoritmos para identificar regiões em tempo $O(W \times H)$ e funciona corretamente com qualquer uma das três estratégias de preenchimento, confirmando a generalidade do design implementado.